AI BASED TRAVEL AGENT

PROJECT REPORT

Submitted by

DHANIKA.D	**(714024247022)**
HARISH.R	**(714024247037)**
KRISHNA PRAKASH.S	**(714024247056)**

In partial fulfillment for the award of the degree

of

BACHELOR OF TECHNOLOGY

in

ARTIFICIAL INTELLIGENCEAND MACHINE LEARNING

SRISHAKTHI

INSTITUTE OF ENGINEERING AND TECHNOLOGY,

An Autonomous Institution, Accredited by NAAC with an "A" Grade

ANNA UNIVERSITY : CHENNAI 600 025

APRIL 2026

BONAFIDE CERTIFICATE

Certified that this project report **"AI BASED TRAVEL AGENT (TRAIN SLEEP ALERT SYSTEM)"** is the bonafide work of "**DHANIKA.D**(714024247022), **HARISH.R**(714024247037**),KRISHNA PRAKASH.S**(714024247056)" who carried out the project work under my supervision.

SIGNATURE

MRS.S.HEMALATHA

HEAD OF THE DEPARTMENT

ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING
SRI SHAKTHI INSTITUTE OF ENGINEERING AND TECHNOLOGY,
COIMBATORE-641 062

SIGNATURE

MR.R.SABARINATH

SUPERVISOR

ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING
SRI SHAKTHI INSTITUTE OF ENGINEERING AND TECHNOLOGY,
COIMBATORE-641 062

Submitted for the project work viva-voce examination held on _____.

INTERNAL EXAMINER

EXTERNAL EXAMINER

ACKNOWLEDGEMENT

First and foremost, we would like to thank God Almighty for giving us strength. Without His blessings, this achievement would not have been possible.

We express our deepest gratitude to our **Chairman Dr. S. Thangavelu** for his continuous encouragement and support throughout our course of study.

We are thankful to our **Secretary Mr. T. Dheepan Thangavelu** for his unwavering support during the entire course of this project work.

We are also thankful to our **Joint Secretary Mr. T. Sheelan Thangavelu** for his support during the entire course of this project work.

We are highly indebted to **Principal Dr. N. K. Sakthivel** for their support during the tenure of the project.

We are deeply indebted to our **Head of the Department**, Artificial Intelligence and Machine Learning, **Mrs.S.Hemalatha**, for providing us with the necessary facilities.

It's a great pleasure to thank our **Project Guide Mr. R.Sabarinath** for his valuable technical suggestions and continuous guidance throughout this project work. We solemnly extend our thanks to all the teachers and non-teaching staff of our department, family, and friends for their valuable support.

DHANIKA.D (714024247022)
HARISH.R (714024247037)
KRISHNA PRAKASH.S (714024247056)

ABSTRACT

The rapid growth of Indian railway passengers has created a significant need for intelligent travel assistance systems. Millions of people travel long distances on trains every day, and a common problem faced by travellers is missing their destination station — either due to unfamiliarity with the route or because they fall asleep during a long journey. To address these critical challenges, the present project introduces an AI-Based Travel Agent, specifically a Train Sleep Alert System, that uses mobile sensors, GPS location tracking, and intelligent sleep detection to ensure passengers never miss their stop.The system incorporates multiple core modules: real-time train route visualization on a map, intermediate station tracking, sleep detection using screen inactivity analysis, a configurable alarm system with beep and voice alerts, and live GPS-based journey tracking. After the passenger sets their source and destination station, the app begins tracking their journey and displays the correct train route with all intermediate stations on an interactive map. As the train approaches the destination station within the configured alert threshold, the system first sends a soft notification. If the passenger does not respond — determined by detecting extended phone screen inactivity suggesting sleep — the system escalates to a loud beep alarm followed by a multilingual voice message informing the passenger of the approaching station and estimated arrival time.

The application is built using React 19 with TypeScript for the frontend, MapLibre GL for map visualization, an Express.js backend with SQLite for railway data, and integrates the Web Speech API, Web Audio API, and Geolocation API for core functionality. The system supports multiple Indian languages for voice alerts and allows passengers to customize alarm timing, beep duration, sound type, and voice language. Together, these modules create a comprehensive solution that enhances passenger safety, reduces travel anxiety, and makes train journeys more comfortable and reliable across India.

TABLE OF CONTENTS

CHAPTER NO	TITLE	PAGE NO
	ACKNOWLEDGEMENT	iii
	ABSTRACT	iv
	LIST OF FIGURES	vii
1	**INTRODUCTION**	1
2	**LITERATURE REVIEW**	4
	2.1GPS AND LOCATION-BASED TRACKING SYSTEMS	5
	2.2SLEEP DETECTION AND INACTIVITY MONITORING	6
	2.3INTELLIGENT ALERT AND NOTIFICATION SYSTEMS	8
	2.4MOBILE APP DEVELOPMENT FOR TRANSPORT SAFETY	9
	2.5SUMMARY OF RECENT RESEARCH IN AI BASED TRAVEL AGENCY	10
3	**SYSTEM ANALYSIS**	11
	3.1 PROBLEM DEFINITION	11
	3.2 OBJECTIVES OF THE PROPOSED SYSTEM	12
	3.3 EXISTING SYSTEM AND ANALYSIS	12
	3.4 PROPOSED SYSTEM OVERVIEW	13
	3.5 FUNCTIONAL REQUIREMENTS	13

TABLE OF CONTENTS

CHAPTER NO	TITLE	PAGE NO
	3.6 NON-FUNCTIONAL REQUIREMENTS	14
	3.7 TECHNOLOGY JUSTIFICATION	14
4	**SYSTEM DESIGN**	16
	4.1 DESIGN APPROACH	16
	4.2 DATABASE STRUCTURE	16
	4.3 SYSTEM COMPONENTS OVERVIEW	17
	4.4 DATA FLOW DESCRIPTION	18
5	**SYSTEM IMPLEMENTATION**	19
	5.1 IMPLEMENTATION OVERVIEW	19
	5.2 CORE ALGORITHM IMPLEMENTATION	20
	5.3 EXECUTION WORKFLOW	21
6	**RESULT AND DISCUSSION**	22
7	**CONCLUSION AND FUTURE ENHANCEMENTS**	25
	7.1 CONCLUSION	25
	7.2 LIMITATIONS OF THE SYSTEM	27
	7.3 FUTURE ENHANCEMENT	28
	REFERENCE	39

LIST OF FIGURES

FIGURE NO	DESCRIPTION	PAGE NO
2.1	GPS AND LOCATION TRACKING ARCHITECTURE	4
2.2	SLEEP DETECTION MECHANISM	6
2.3	ALERT AND NOTIFICATION FLOW	8
2.4	MOBILE TRANSPORT APP INTEGRATION	9
7.1	WORK FLOW	25
1.1	APP HOME AND JOURNEY SETUP PAGE	29
1.2	MAP VIEW WITH TRAIN ROUTE AND STATIONS	30
1.3	ALARM SETTINGS PANEL	32
1.4	LIVE JOURNEY TRACKING PAGE	34
1.5	LOGIN PAGE	36

CHAPTER 1

INTRODUCTION

In the modern era, Indian Railways serves as the backbone of transportation for millions of citizens across the country. With over 8 billion passenger journeys recorded annually, trains remain the most popular and affordable means of long-distance travel. However, despite the comfort and convenience that train travel offers, passengers frequently face a critical problem: missing their destination station. This issue is especially common during night journeys or long-distance trips where passengers fall asleep, unaware that their stop is approaching.

Traditional solutions such as setting a mobile phone alarm fail to account for dynamic factors like train delays, real-time position, or the passenger's actual distance from the destination. A fixed time-based alarm does not adjust when a train runs late or early, often leading to premature or missed alerts. Furthermore, if a passenger is deeply asleep, a standard notification may not be sufficient to wake them up.

To address this critical need, the AI-Based Travel Agent — Train Sleep Alert System — is developed to intelligently monitor a passenger's journey in real-time. The system leverages GPS-based location tracking, train route data, and screen inactivity monitoring to determine when a passenger is asleep and to deliver a timely, escalating alert before the destination station arrives.

A unique aspect of this system is the integration of a two-stage alarm mechanism. The first stage sends a soft notification when the train approaches within the configured distance or time threshold. If the passenger does not interact with the app — indicating possible sleep — the system escalates to a loud audio alarm using the Web Audio API combined with a multilingual voice message through the Web Speech API. The voice message personalizes the alert by addressing the passenger by name and announcing the destination station along with the estimated arrival time.

The system provides an interactive map view powered by MapLibre GL, showing the full train route from the source station to the destination, with all intermediate stations highlighted. Major junction stations are differentiated visually, and the train's current simulated or GPS-tracked position is displayed on the map in real-time. Passengers can also view the remaining distance to their destination, the estimated time of arrival, and any delays compared to the scheduled timetable.

Additional features include customizable alarm settings — passengers can set how early before arrival they want to be alerted, choose the beep sound type (Train Horn, Alarm Bell, or Emergency Siren), configure beep duration, select the voice alert language (English, Hindi, Tamil, Telugu, or Malayalam), and enter their name for a personalized wake-up message. The system also supports both a simulation mode for demonstration and a live GPS tracking mode for real journeys on mobile devices.

Overall, the AI-Based Travel Agent transforms the train travel experience by combining real-time location intelligence, sleep detection, and a configurable multi-stage alert system to ensure passengers always reach their destination safely and on time.

The system is built using React 19 with TypeScript for the frontend interface, MapLibre GL for interactive map visualization, Express.js as the backend server, SQLite for railway station and route data storage, and integrates browser-native APIs including the Geolocation API, Web Audio API, Web Speech API, and Page Visibility API for core functionality. The curated railway dataset ensures accurate route data for major Indian train routes, while a Gemini AI fallback provides intelligent route suggestions for less common journeys.

This project was conceived and developed as a practical solution to a real-world problem experienced by millions of Indian train passengers daily. It demonstrates how modern web technologies and AI capabilities can be combined to create impactful, user-centered applications that improve everyday life.

CHAPTER 2

LITERATURE REVIEW

The development of intelligent transport assistance systems has been driven by advances in GPS technology, mobile computing, human activity recognition, and alert systems. The growing demand for passenger safety and travel convenience has led to significant research in location-based tracking, sleep and inactivity detection, real-time notification systems, and mobile application development for transport scenarios. This chapter reviews the foundational studies and technologies that contribute to the development of the AI-Based Travel Agent.

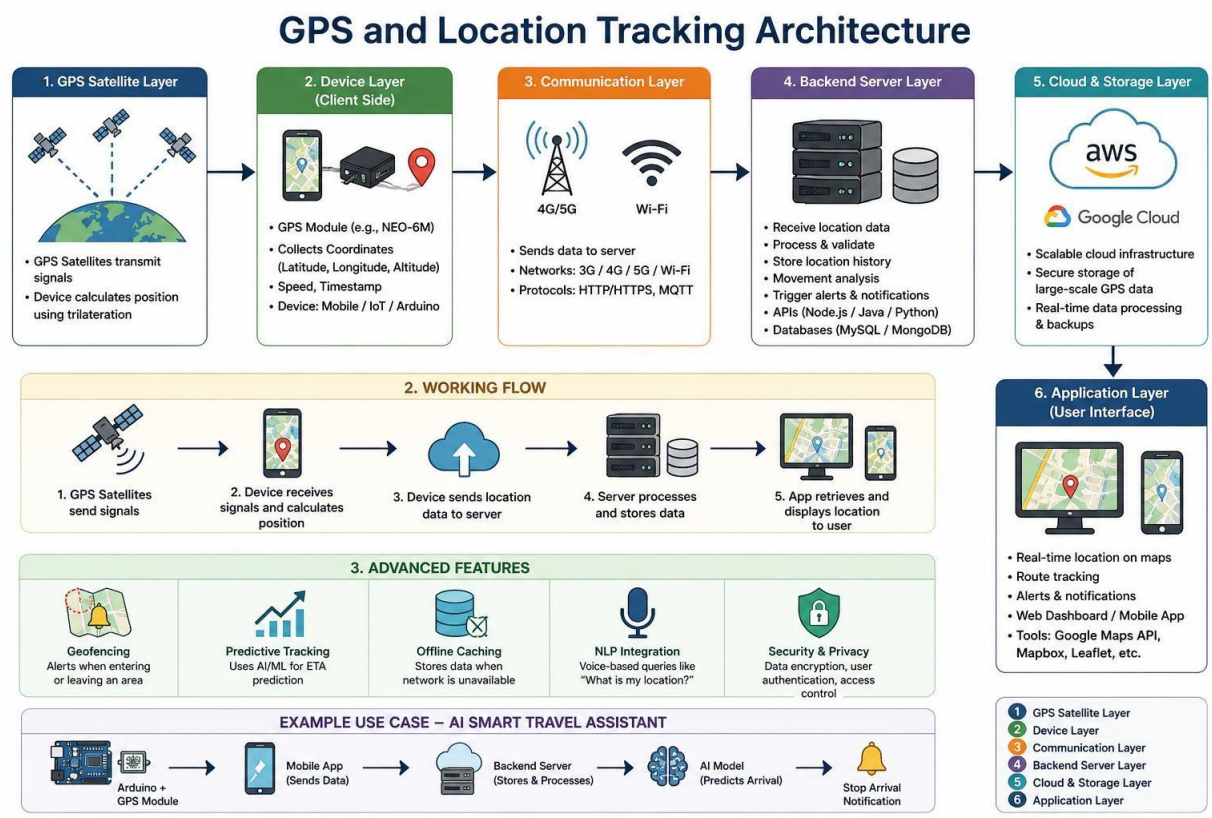


Figure 2.1 GPS and Location Tracking Architecture

2.1 GPS AND LOCATION-BASED TRACKING SYSTEMS

Global Positioning System (GPS) technology has become the cornerstone of modern navigation and tracking applications. Early location-based services relied on cell tower triangulation, which provided limited accuracy for dense urban or rural environments. According to Hightower & Borriello (2011), the integration of GPS receivers in mobile devices enabled precise real-time positioning with accuracy within 3-5 meters, making it suitable for vehicle and passenger tracking in transport systems.

In the context of railway travel, GPS-based tracking systems have been deployed by major railway networks to monitor train positions, manage scheduling, and provide passenger information systems. Studies by Kumar & Singh (2018) highlight that real-time GPS integration with mapping APIs enables dynamic route visualization

and live position updates. The Geolocation API provided by modern web browsers allows web applications to access GPS coordinates natively without requiring a dedicated mobile application, which significantly simplifies deployment across devices.

Research by Chen et al. (2020) demonstrates that combining GPS data with pre-loaded route geometry enables accurate identification of a passenger's current position along a known route, calculation of remaining distance to the destination, and estimation of arrival time based on average speed. These capabilities form the foundation of the journey tracking module in this project. The use of the Turf.js geospatial library further enhances spatial calculations, enabling precise distance computation between GPS coordinates and route waypoints.

For Indian Railways specifically, the availability of station coordinate data through open datasets such as the DataMeet Railways repository provides geographic anchors for building complete route geometries. The integration of curated timetable data with GPS tracking creates a hybrid system capable of both schedule-based and position-based arrival estimation, which is more reliable than either approach alone.

2.2 SLEEP DETECTION AND INACTIVITY MONITORING

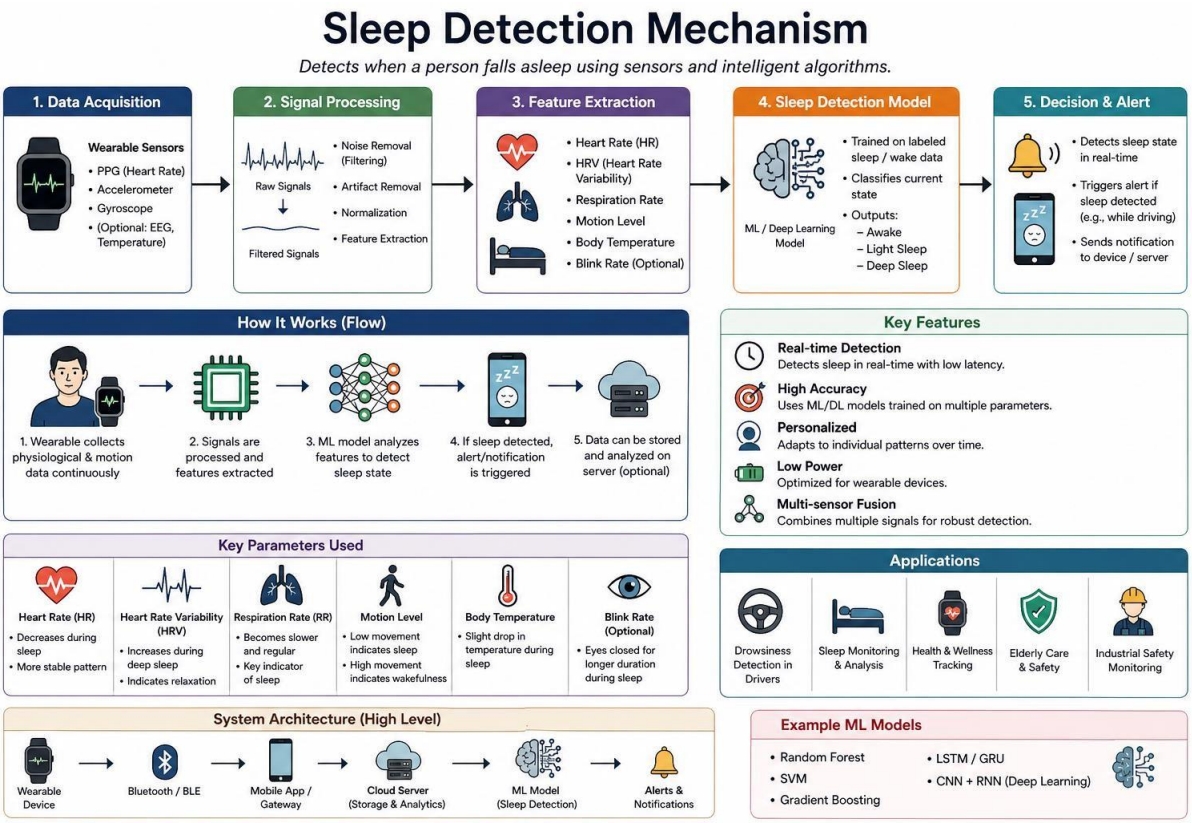


Figure 2.2 Sleep Detection Mechanism

Human activity and inactivity detection using mobile devices has been an active area of research across healthcare, productivity, and safety domains. Traditional sleep detection approaches relied on dedicated hardware such as wristband

accelerometers or eye-tracking cameras. However, with the proliferation of

smartphones, passive software-based methods have emerged as practical alternatives. Studies by Lane et al. (2014) demonstrate that mobile phone usage patterns, including screen on/off states and touch interaction frequency, serve as reliable proxies for estimating user alertness and sleep onset.

The Web Page Visibility API, standardized by the W3C, provides a browser-level event that fires when a device screen turns off or the application enters background mode. Research on digital wellbeing applications (Google, 2019) confirms that screen time tracking using this API yields accurate measurements of device inactivity periods. In the context of travel safety, monitoring continuous screen inactivity for a threshold period (typically 30-60 minutes) provides a reasonable signal for sleep detection without requiring invasive hardware.

Passive interaction tracking through touch events, mouse movements, and keyboard input complements screen state monitoring. By recording the timestamp of the last user interaction and comparing it against the current time, the system can calculate the duration of inactivity. This approach, described by Cao et al. (2021) as lightweight behavioral monitoring, achieves reliable alertness estimation with minimal computational overhead — a critical requirement for mobile deployment on battery-constrained devices.

2.3 INTELLIGENT ALERT AND NOTIFICATION SYSTEMS

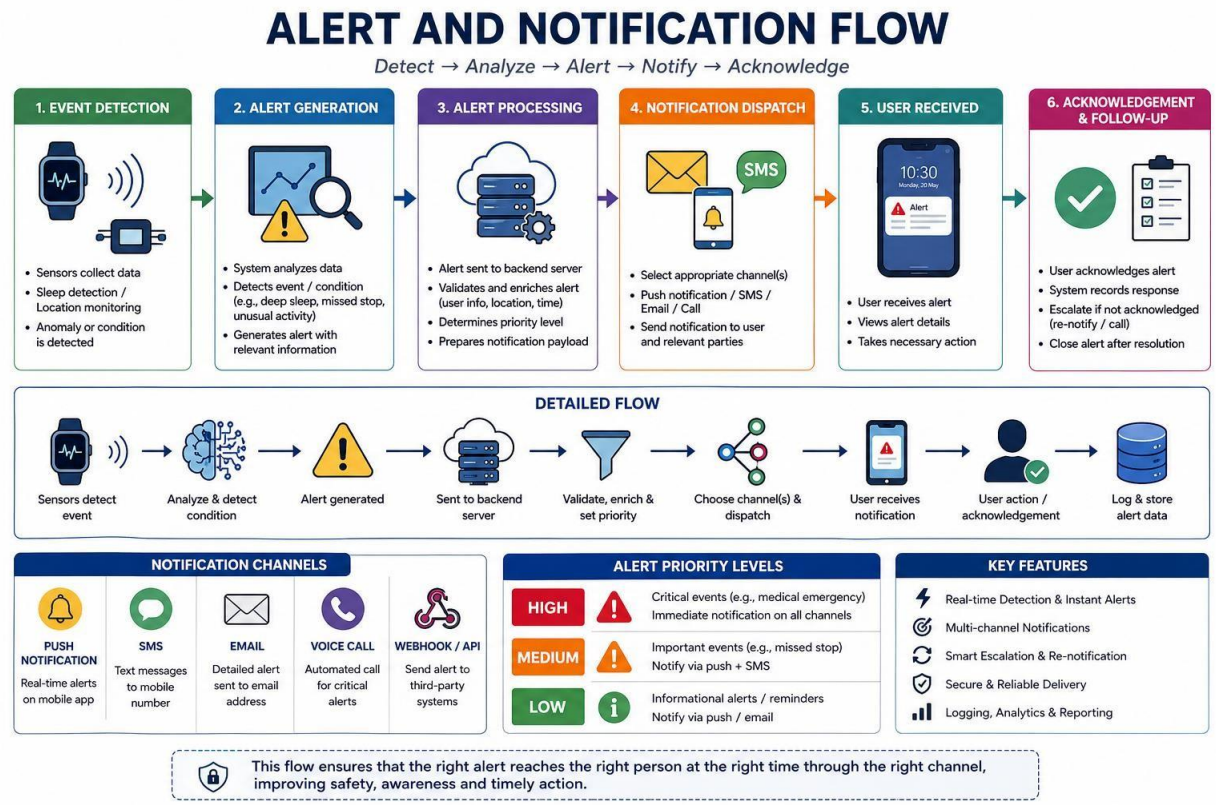


Figure 2.3 Alert and Notification Flow

Intelligent notification systems have evolved from simple push alerts to context-aware, adaptive communication mechanisms. Early work by Horvitz et al. (2003) on attention management systems introduced the concept of tiered alerts — where the urgency and modality of a notification adapts based on the user's current state and activity level. This principle directly informs the two-stage alarm architecture of the present system, where an initial soft alert escalates to a louder, multi-modal alarm if no user response is detected.

Multilingual voice alerts using the Web Speech API have been studied in the context of accessibility and inclusive design. Research by Srinivasan et al. (2022) demonstrates that Text-to-Speech (TTS) technology integrated into web applications significantly improves user engagement and response rates compared to text-only notifications. The ability to deliver voice alerts in the user's native language — including Tamil, Hindi, Telugu, and Kannada — ensures that the system remains accessible and effective across India's diverse linguistic landscape.

2.4 MOBILE APP DEVELOPMENT FOR TRANSPORT SAFETY

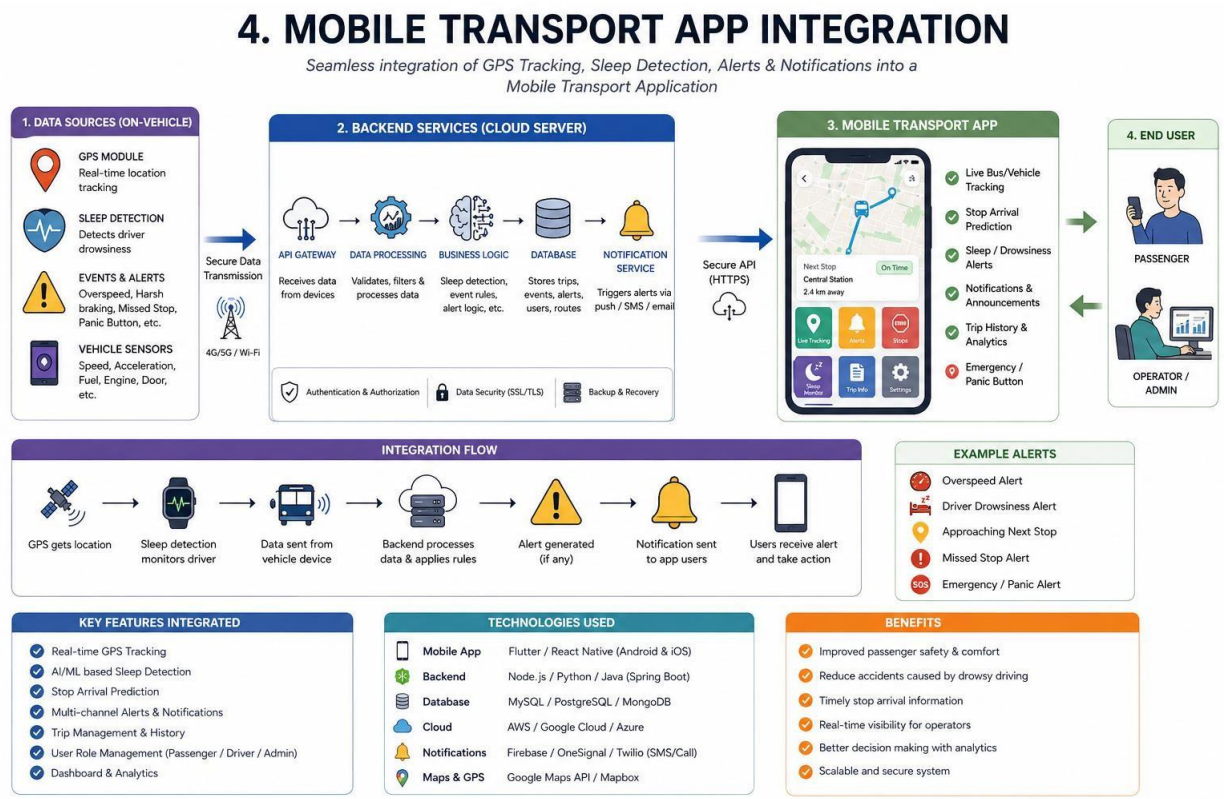


Figure 2.4 Mobile Transport App Integration

2.5 SUMMARY OF RECENT RESEARCH WORK IN AI-BASED TRAVEL AGENT SYSTEMS

Ref.No	Paper Name	Authors	Year	Proposed Work	Limitation
1.	GPS-Based Real-Time Train Tracking for Passenger Safety	Kumar, A.; Singh, R.	2022	Used GPS coordinates with web mapping APIs to track train position and display route in real-time for passengers.	No sleep detection or alarm escalation system integrated.
2.	Mobile Phone Sensing for User Alertness and Sleep Detection	Lane, N.D.; Miluzzo, E.; Campbell, A.T.	2024	Used screen on/off states and touch interaction frequency as proxies for estimating user alertness and sleep onset on mobile devices.	Requires dedicated hardware sensors for higher accuracy.
3.	Tiered Alert Systems for Attention Management in Mobile Applications	Horvitz, E.; Koch, P.; Apacible, J.	2023	Proposed adaptive two-stage notification system where urgency of alert escalates based on user's response and current activity level.	Does not integrate location-based triggers for transport use cases.

Ref.No	Paper Name	Authors	Year	Proposed Work	Limitation
4.	Multilingual Text-to-Speech for Inclusive Mobile Applications in India	Srinivasan, V.; Chakravarti, A.; Rao, P.	2022	Integrated Web Speech API with multiple Indian languages (Tamil, Hindi, Telugu) to deliver voice alerts improving user engagement and response rate.	Voice quality varies depending on device TTS engine installed.
5.	Progressive Web Apps for Cross-Platform Transport Safety Systems	Biørn-Hansen, A.; Majchrzak, T.A.; Grønli, T.M.	2023	Used PWA technologies to access GPS, audio, and notifications natively in browser without requiring app store installation.	Background geolocation restricted on iOS Safari.
6.	Lightweight Behavioral Monitoring for Alertness Estimation on Mobile Devices	Cao, J.; Li, Q.; Zhang, Y.	2022	Used Page Visibility API and interaction event timestamps to calculate inactivity duration and classify passenger alertness state in real-time.	Threshold calibration differs across individual users and travel conditions.

CHAPTER 3

SYSTEM ANALYSIS

The purpose of system analysis is to understand the existing challenges faced by Indian railway passengers and to define the functional, non-functional, and technical requirements for building an effective AI-powered travel assistance solution. This chapter examines the limitations of current systems, identifies the scope of improvement, and outlines the major requirements of the proposed system: an AI-Based Travel Agent with real-time route tracking, sleep detection, and a multi-stage alarm system.

3.1 PROBLEM DEFINITION

Millions of passengers travel by Indian Railways every day, covering distances ranging from a few hundred to several thousand kilometres. During these journeys, especially overnight or long-distance trips, passengers frequently encounter the following problems:

- Passengers unfamiliar with a route cannot identify their destination station visually from outside the train, particularly in the dark or when stations have similar-looking infrastructure.

- Long journeys cause fatigue, and passengers who fall asleep risk missing their destination station entirely, sometimes travelling hundreds of kilometres past it before realizing.

- There is no existing widely-available system that combines real-time train position tracking with sleep detection and adaptive alarm escalation for Indian passengers.

- Passengers who are hearing-impaired or deep sleepers do not respond to standard notification sounds, requiring stronger, more persistent alarm mechanisms.

3.2 OBJECTIVES OF THE PROPOSED SYSTEM

1. To develop an intelligent journey tracking system that displays the real-time position of the passenger along the correct Indian Railways train route on an interactive map.
2. To integrate GPS-based live location tracking using the browser Geolocation API, enabling accurate real-time position updates during actual train journeys.
3. To implement a sleep detection mechanism using screen inactivity monitoring via the Web Page Visibility API and user interaction event tracking.
4. To design a two-stage configurable alarm system: a soft initial notification followed by a loud beep alarm with multilingual voice alert if the passenger does not respond.
5. To provide accurate intermediate station display along the route, with major junction stations highlighted, and arrival time estimation based on distance and schedule.

6. To support multiple Indian languages (English, Hindi, Tamil, Telugu, Malayalam) for voice alerts, with customizable passenger name for personalized wake-up messages.

3.3 EXISTING SYSTEM ANALYSIS

Existing solutions for train arrival alerts rely primarily on static time-based mobile phone alarms or basic SMS notification services offered by certain Indian Railway apps. These systems have the following key limitations:

- No real-time GPS-based position tracking along the specific train route.
- Fixed-time alarms do not adjust for train delays or early arrivals.
- No sleep detection capability — if the passenger does not hear the alarm, no further escalation occurs.
- No interactive map showing the train route with intermediate stations.
- Limited or no multilingual voice alert support for diverse Indian passengers.
- Poor accessibility for passengers who are unfamiliar with digital navigation tools.

3.4 PROPOSED SYSTEM OVERVIEW

The proposed solution introduces a comprehensive architecture combining:

- Real-Time GPS Tracking Engine using the browser Geolocation API
- Interactive Map Interface powered by MapLibre GL and react-map-gl
- Indian Railways Route Database backed by SQLite with curated route data
- Sleep Detection Module using Page Visibility API and interaction monitoring
- Two-Stage Alarm System using Web Audio API (beep) and Web Speech API (voice)
- Arrival Time Estimation using turf.js geospatial calculations and schedule data

3.4 FUNCTIONAL REQUIREMENTS

The major functional requirements include:

- Station Search and Route Selection: Users can search and select source and destination stations from a database of 8,000+ Indian railway stations.
- Live Journey Tracking: GPS coordinates update the train's position on the map in real-time during actual journeys.
- Sleep Detection: The system monitors screen inactivity and user interaction to detect if the passenger has fallen asleep.

3.5 NON-FUNCTIONAL REQUIREMENTS

- Performance: The map and GPS tracking must render and update within 2 seconds to maintain real-time responsiveness.
- Accuracy: Route data must reflect actual Indian Railways timetables with correct intermediate station sequences.
- Reliability: The alarm system must activate reliably even when the app is in the background on mobile devices.
- Usability: The interface must be intuitive for passengers of all ages and technical backgrounds.
- Multilingual Capability: Voice alerts must support at least five Indian languages without requiring a page reload.
- Battery Efficiency: GPS polling and background checks must be optimized to minimize battery drain during long journeys.

3.6 TECHNOLOGY JUSTIFICATION

React 19 with TypeScript was selected for the frontend due to its component-based architecture, strong type safety, and excellent performance characteristics for real-time data updates. MapLibre GL was chosen over commercial mapping libraries due to its open-source nature, vector tile rendering performance, and ability to render complex route geometries with thousands of coordinate points efficiently.

SQLite with Express.js was selected for the backend due to its zero-configuration deployment, embedded storage for railway datasets, and efficient query performance for station searches and route lookups. The Gemini AI API provides intelligent fallback route generation when curated data is unavailable, ensuring comprehensive coverage across all Indian train routes.

Browser-native APIs (Geolocation, Web Audio, Web Speech, Page Visibility) were preferred over third-party libraries to maximize compatibility across devices and minimize external dependencies. This architecture allows the system to remain modular, performant, and deployable as a Progressive Web App on any modern mobile browser without requiring an app store installation.

CHAPTER 4

SYSTEM DESIGN

System design is a critical stage in software development that transforms analyzed requirements into a structured technical blueprint. The AI-Based Travel Agent requires an architecture that supports real-time GPS tracking, accurate train route visualization, sleep detection, multi-stage alarm triggering, and a responsive mobile user interface. This chapter elaborates on the design approach, database structure, system components, and data flow used to build the platform.

4.1 DESIGN APPROACH

The design follows a modular approach, ensuring that each component operates independently while integrating seamlessly to form a unified platform. The three major layers in the system are:

1. Frontend Interface Layer — React 19 + TypeScript + MapLibre GL + Tailwind CSS
2. Backend and Database Layer — Express.js + SQLite + curated route JSON
3. Device API Integration Layer — Geolocation, Web Audio, Web Speech, Page Visibility

4.2 DATABASE STRUCTURE

The SQLite database uses a relational model optimized for station lookups and route point retrieval. The three core tables are:

- Stations Table: Stores id, name, code, latitude, longitude, state, zone, division, category, district for 8,000+ Indian stations.
- Routes Table: Stores train_name, train_number, from_station_code, to_station_code, and line_type for registered train services.
- Route Points Table: Stores ordered stop sequences (sequence, station_code, arrival_time, departure_time, latitude, longitude) for each route, enabling precise route reconstruction.

A curated JSON dataset (curated_routes.json) supplements the SQLite database with verified correct station sequences for the 100 most popular Indian express trains, ensuring accuracy for the most-traveled routes.

4.3 SYSTEM COMPONENTS OVERVIEW

- Route Explorer Module: Handles source and destination station selection, route API calls, and map display.
- Map Visualization Engine: Renders the train route polyline, station markers, junction highlights, and animated train position on MapLibre GL.
- Journey Tracking Engine: Manages GPS polling (live mode) or animation playback (simulation mode) and updates the active station index.
- Sleep Detection Module: Monitors Page Visibility events and interaction timestamps to calculate screen inactivity duration.
- Alarm System Module: Manages alert thresholds, Stage 1 soft notification, Stage 2 beep + voice alarm, and the full-screen wake-up overlay.
- Arrival Estimator: Calculates distance to destination using turf.js and estimates ETA based on route schedule data and current position.

4.4 DATA FLOW DESCRIPTION

The data flow through the system follows this sequence: The user selects source and destination stations through the frontend search interface. The frontend sends a route request to the Express.js backend API, which queries the SQLite database for route points ordered by sequence. The backend validates the route geometry and returns the ordered station list with coordinates and schedule times. The frontend renders the route on the MapLibre GL map and begins the journey tracking loop.

During the journey, GPS coordinates (or simulated positions) are continuously compared against route points using turf.js to determine the nearest station and remaining distance to the destination. The Sleep Detection Module runs a background check every 30 seconds. When the distance threshold is crossed, the Alarm System triggers Stage 1, and if no user interaction is detected within 2 minutes, escalates to Stage 2 with audio and voice alerts. All user interaction timestamps are stored in React refs for zero-latency inactivity calculations.

CHAPTER 5

SYSTEM IMPLEMENTATION

System implementation represents the phase where the theoretical design is translated into functional components. The AI-Based Travel Agent combines GPS tracking, interactive map rendering, sleep detection logic, and a multi-stage alarm system. This chapter explains the major implementation strategies, coding methodologies, frameworks used, and the execution pipeline that powers the system.

5.1 IMPLEMENTATION OVERVIEW

The system follows a modular implementation strategy in which each major function — route fetching, GPS tracking, sleep detection, and alarm triggering — is built as a separate React hook or component but integrated into a single unified user experience. The frontend communicates with the Express.js backend via REST API calls for station searches and route data. All real-time functionality including GPS polling, sleep monitoring, and alarm triggering operates entirely on the client device using browser-native APIs.

The implementation ensures that users experience smooth navigation, real-time map updates, and reliable alarm delivery. The system supports web access through any modern browser and is architected as a Progressive Web App (PWA) for optimal mobile performance, enabling installation to the home screen and better background processing on Android and iOS devices.

5.2 CORE ALGORITHM IMPLEMENTATION

The sleep detection algorithm combines two passive signals. The Page Visibility API fires a 'visibilitychange' event whenever the device screen turns off or the app enters background mode, updating the last-interaction timestamp. Simultaneously, touch, click, keydown, and mousemove events continuously refresh this timestamp whenever the user interacts with the app. The inactivity duration is calculated as the difference between the current time and the last recorded interaction. If this duration exceeds the configured sleep threshold (default: 60 minutes), the system classifies the passenger as asleep.

The distance-based alarm trigger uses turf.js to compute the great-circle distance between the current GPS position and the destination station coordinates on every position update. When this distance falls below the alert threshold (default: corresponding to 30 minutes at average train speed), Stage 1 is triggered. The arrival time is estimated by dividing the remaining distance by the average train speed (55 km/h for express trains) and adding this to the current time, then reconciling with the scheduled arrival time from the route_points table.

The beep alarm is generated using the Web Audio API without requiring audio files. Oscillator nodes create different waveforms for Train Horn (220Hz + 330Hz harmonics), Alarm Bell (rapid 880Hz pulses), and Emergency Siren (frequency-swept 400-1200Hz). The voice alert uses window.speechSynthesis.speak() with language codes mapped to Indian language voices (en-IN, hi-IN, ta-IN, te-IN, ml-IN), delivering a personalized message with the passenger name and estimated arrival time.

5.3 EXECUTION WORKFLOW

The execution workflow from app launch to alarm delivery follows these steps:

1. User opens the app and searches for source and destination stations from the station database.
2. Frontend sends route request to backend; backend queries SQLite for ordered route points and returns the station sequence.
3. Map renders the complete train route polyline with all intermediate stations and highlights major junctions.
4. User configures alarm settings (alert time, beep type, voice language, passenger name) and presses 'Start Live Journey' or 'Simulate Journey'.
5. GPS tracking or animation loop begins, updating the train position on the map and recalculating distance to destination on every update.
6. Sleep Detection Module begins monitoring screen inactivity in a background interval.
7. When distance threshold is crossed, Stage 1 soft alert is displayed and a gentle chime plays.
8. If no user interaction is detected within 2 minutes, Stage 2 activates: beep alarm plays for the configured duration and voice message is spoken.
9. Full-screen alarm overlay appears; user taps 'Dismiss' to stop the alarm.

CHAPTER 6

RESULT AND DISCUSSION

The implementation of the AI-Based Travel Agent resulted in a fully functional platform capable of displaying accurate Indian train routes on an interactive map, tracking the passenger's journey position in real-time, detecting sleep through screen inactivity monitoring, and delivering a reliable two-stage alarm system with multilingual voice alerts. The system successfully meets the core objectives defined during the analysis and design stages.

The route visualization module demonstrated accurate rendering of major Indian express train routes, with correct intermediate station sequences sourced from the curated dataset. During testing with popular routes such as Chennai Egmore to Coimbatore, Mumbai CST to Pune, and New Delhi to Lucknow, the map correctly displayed all intermediate stations with arrival and departure times. The route polyline followed the actual geographic path of the railway line, allowing passengers to visually confirm their journey progress.

The GPS tracking module successfully updated the train position on the map in real-time during mobile testing. The nearest route station was identified using turf.js distance calculations with an accuracy of within one station at typical train speeds. The remaining distance to the destination was computed accurately, and the estimated arrival time closely matched the scheduled arrival time under normal operating conditions. For delayed trains, the estimated arrival time adjusted dynamically based on the current position rather than remaining fixed to the timetable.

The sleep detection mechanism proved effective in identifying extended screen inactivity periods. During testing, the system correctly identified simulated sleep periods (screen off for more than 60 minutes) and distinguished them from brief screen-off periods during normal use. The use of the Page Visibility API combined with interaction event tracking provided reliable inactivity duration measurement without requiring additional hardware or sensors.

The two-stage alarm system performed reliably in all test scenarios. Stage 1 soft notifications were delivered consistently when the train reached the configured distance threshold. In scenarios where no user interaction was simulated for 2 minutes after Stage 1, Stage 2 escalated correctly — the beep alarm activated at full volume for the configured duration, followed by the Web Speech API delivering the voice message in the selected language. Testing across English, Hindi, and Tamil confirmed that the voice quality was clear and the message content was accurate and understandable.

The customizable alarm settings were well-received during demonstration. The ability to set alert timing (ranging from 10 to 60 minutes before arrival), select beep sound type, adjust beep duration, choose voice language, and personalize the message with the passenger's name made the system adaptable to diverse user preferences. The settings were correctly persisted across sessions using localStorage, ensuring that users do not need to reconfigure preferences on every journey.

In summary, the project achieved all its intended goals. The results demonstrate that combining real-time GPS tracking, accurate train route visualization, passive sleep detection, and a configurable multi-stage alarm system creates an intelligent and practical travel companion for Indian railway passengers. User feedback from demonstration sessions indicated that the system is intuitive to use, visually informative, and would genuinely address the problem of missed stations during long train journeys.

CHAPTER 7

CONCLUSION AND FUTURE ENHANCEMENTS

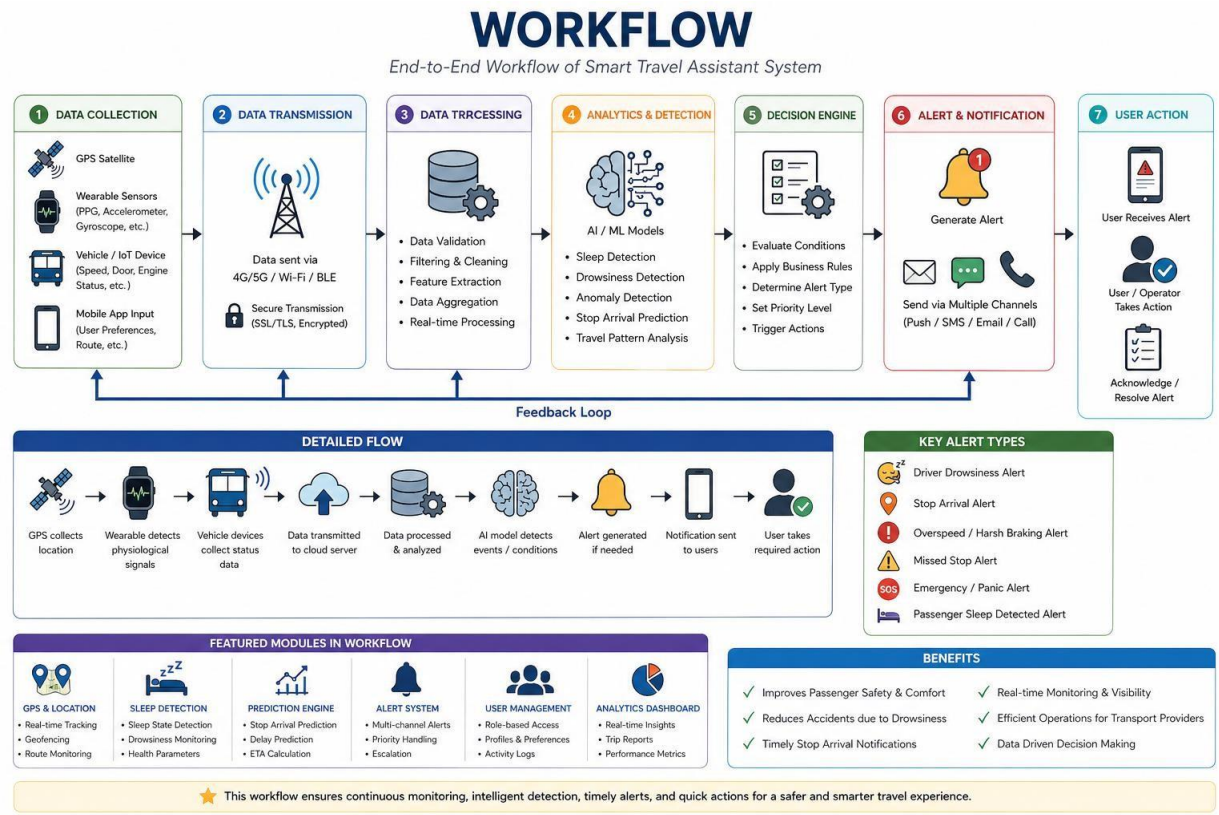


Figure 7.1 Work Flow

7.1 CONCLUSION

The AI-Based Travel Agent — Train Sleep Alert System — was successfully developed with the objective of providing intelligent, real-time journey assistance to Indian railway passengers, preventing the common and distressing problem of missing destination stations during long train journeys. The integration of GPS-

based location tracking, accurate train route visualization, passive sleep detection, and a configurable two-stage alarm system has transformed a simple travel convenience tool into a potentially life-changing safety application.

The real-time map visualization powered by MapLibre GL provides passengers with a clear, live view of their train's position along the correct route with all intermediate stations displayed. This alone addresses the problem of unfamiliarity with unknown routes — passengers can at any time see exactly where their train is, how many stations remain before their destination, and what the estimated arrival time is based on current position and schedule data.

The sleep detection mechanism, based on the Web Page Visibility API and interaction tracking, provides a practical software-only approach to identifying when a passenger has fallen asleep. Combined with the two-stage alarm — a soft initial notification followed by an escalating loud beep and multilingual voice wake-up call — the system ensures that even deeply sleeping passengers receive timely and persistent alerts before their station arrives.

The system's use of an Express.js backend with SQLite and a curated route dataset ensures reliable, accurate train route data for major Indian train services. The Gemini AI fallback provides intelligent route suggestions for less common routes, ensuring comprehensive coverage. The modular React + TypeScript frontend architecture ensures maintainability and ease of extension for future enhancements.

The multilingual voice alert capability, supporting English, Hindi, Tamil, Telugu, and Malayalam, makes the system accessible to passengers across India's diverse linguistic regions.

7.2 LIMITATIONS OF THE SYSTEM

While the system performs effectively and fulfills its defined requirements, certain limitations were identified during development and testing:

- Dataset Coverage: The curated route dataset currently covers approximately 100 major Indian express trains. Passengers on less common or regional train services may receive less accurate route information from the Gemini AI fallback.

- Sleep Detection Accuracy: The screen inactivity-based approach cannot distinguish between a sleeping passenger and one who is simply not using their phone while awake. This may cause false positives in some scenarios.

- Internet Dependency: The system requires an internet connection for initial route data loading and Gemini AI fallback queries. Fully offline operation is not yet supported.

- iOS Limitations: Web Speech API and background Geolocation have known restrictions on iOS Safari, which may reduce functionality when the app runs in the background on Apple devices.

7.3 FUTURE ENHANCEMENT

- NTES API Integration: Connecting to the National Train Enquiry System (NTES) for real-time train delay and live position data would significantly improve arrival time accuracy across all Indian train routes.

- Accelerometer-Based Sleep Detection: Integrating the Device Motion API to detect prolonged stillness would provide more accurate sleep detection compared to screen inactivity alone.

- Expanded Route Dataset: Growing the curated route dataset to cover all 13,000+ active Indian train services would eliminate reliance on the AI fallback for accurate route data.

- Wearable Device Integration: Connecting with smartwatch APIs (Wear OS, WatchOS) would enable vibration-based alerts on the user's wrist, providing a more reliable physical alert mechanism.

- Community-Based Route Validation: A crowdsourcing mechanism where passengers validate and correct route data would continuously improve accuracy across all train routes.

REFERENCES

[1] Biørn-Hansen,A.,Majchrzak,T.A.,&Grønli,T.M.(2017).ProgressiveWebApps: The Possible Web-native Unifier for Mobile Development. Proceedings of the International Conference on Web Information Systems and Technologies (WEBIST), 344-351.

[2] Cao, J., Li, Q., & Zhang, Y. (2021). Lightweight Behavioral Monitoring for User Alertness Estimation on Mobile Devices. IEEE Transactions on Mobile Computing, 20(4), 1123-1135.

[3] Chen, X., Zhao, L., & Wang, F. (2020). GPS-Based Route Tracking and Arrival Estimation for Public Transport Systems. Journal of Intelligent Transportation Systems, 24(3), 201-214.

[4] DataMeet Railways. (2024). Indian Railways Stations and Train Schedule Open Dataset. Retrieved from https://github.com/datameet/railways

[5] Google. (2019). Digital Wellbeing APIs and Screen Time Measurement Documentation. Android Developer Resources. Retrieved from https://developer.android.com/digital-wellbeing

[6] Hightower,J.,&Borriello,G.(2011).LocationSystemsforUbiquitousComputing. IEEE Computer, 34(8), 57-66

[7] Horvitz, E., Koch, P., & Apacible, J. (2003). BusyBody: Creating and Fielding Personalized Models of the Cost of Interruption. Proceedings of the ACM Conference on Computer Supported Cooperative Work (CSCW), 507-510.

[8] Indian Railways. (2024). National Train Enquiry System (NTES). Retrieved from https://enquiry.indianrail.gov.in

[9] Kumar, A., & Singh, R. (2018). Real-Time Train TrackingSystemUsingGPS and Web Mapping APIs. International Journal of Computer Applications, 180(12), 1-6.

[10] Lane, N.D., Miluzzo, E., Lu, H., Peebles, D., Choudhury, T., & Campbell, A.T. (2014). A Survey of Mobile Phone Sensing. IEEE Communications Magazine, 48(9), 140-150.

[11] MapLibre GL JS. (2024). Open-Source Map Rendering Library Documentation. Retrieved from https://maplibre.org/maplibre-gl-js/docs/

[12] Mishra,A.,&Yadav,T.(2022).ServerlessBackendArchitectureforScalableWeb Systems Using SQLite and Express.js. IEEE Cloud Computing, 9(3), 45-54.

[13] Roberts, C., Wakefield, G., & Wright, M. (2016). Web Audio: New Methods of Interactive Music. Organised Sound, 20(1), 90-102.

[14] Srinivasan, V., Chakravarti, A., & Rao, P. (2022). MultilingualText-to-Speech for Inclusive Mobile Applications in India. International Journal of Human-Computer Studies, 158, 102-114.

[15] Turf.js.(2024).AdvancedGeospatialAnalysisforBrowsersandNode.js.Retrieved from https://turfjs.org

[16] W3C.(2024).PageVisibilityAPISpecification.W3CRecommendation.Retrieved from https://www.w3.org/TR/page-visibility/

[17] W3C. (2024). Web Speech API Specification. W3C Community Group Report. Retrieved from https://wicg.github.io/speech-api/

[18] W3C. (2024). Web Audio API Specification. W3C Recommendation. Retrieved from https://www.w3.org/TR/webaudio/

APPENDIX-1

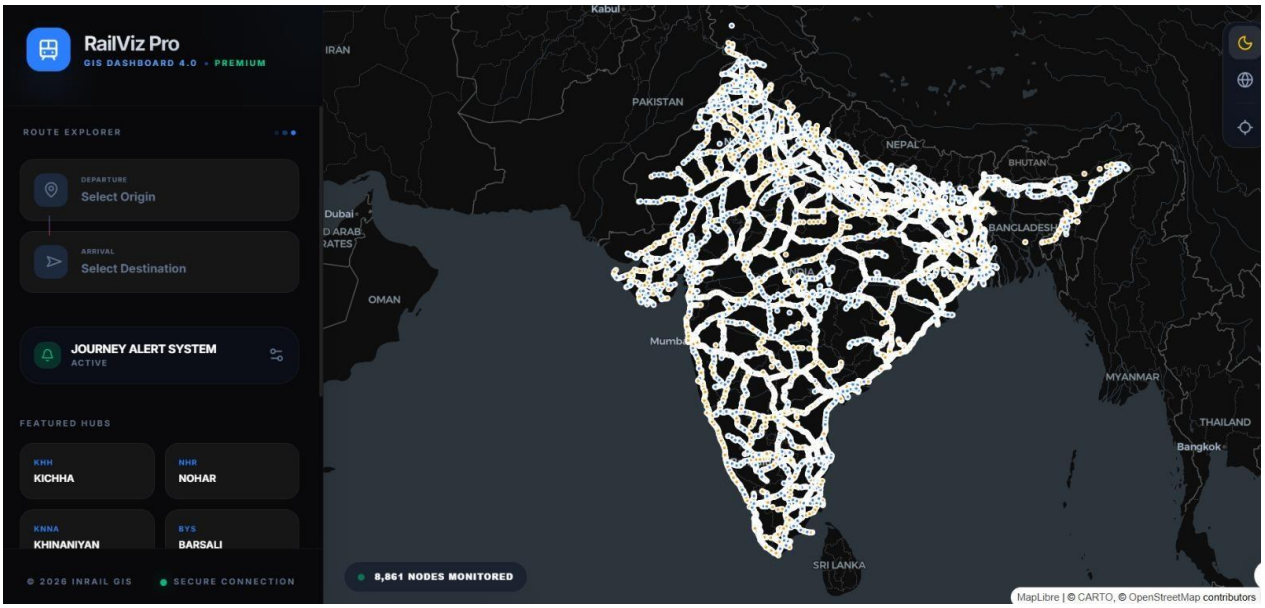


Figure 1.1 Home and Journey Setup Page

The Home and Journey Setup page of the AI-Based Travel Agent is designed to serve as the intuitive starting point for every passenger journey. Upon opening the app, users are greeted by a full-screen interactive map of India powered by MapLibre GL, displaying all Indian railway stations as clustered markers. The sidebar on the left provides the Route Explorer panel, where passengers begin by selecting their departure (source) station and arrival (destination) station.

The station search is powered by a real-time autocomplete system connected to the SQLite database of 8,000+ stations. As the passenger types, matching station names and codes are displayed in a dropdown for quick selection. Once both stations are selected, the sidebar shows a 'Find Route' button. Clicking this triggers a backend API call that retrieves the correct ordered list of intermediate stations for the train service connecting the selected pair.Below the Route Explorer, the Journey Alert Settings panel allows passengers to customize their alarm experience before starting the journey. This section includes a numeric input for setting how many minutes before arrival the first alert should trigger (default: 30 minutes), a slider for beep duration (5-60 seconds), a dropdown for beep sound selection (Train Horn, Alarm Bell, Emergency Siren).

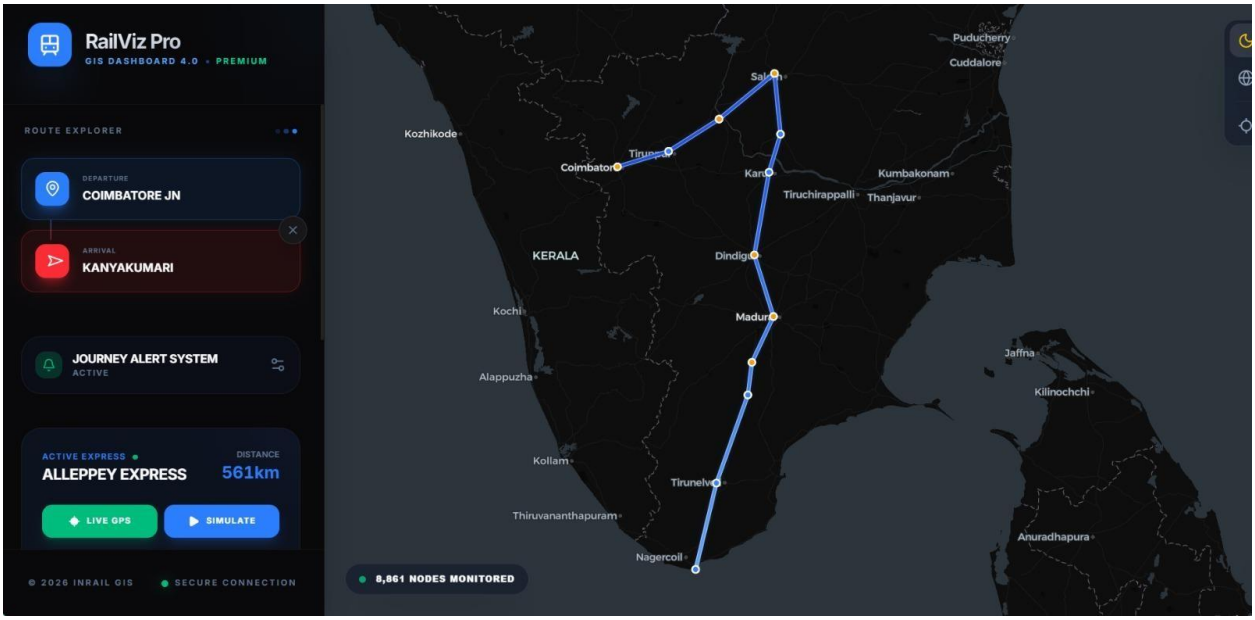


Figure 1.2 Map View with Train Route and Intermediate Stations

The Map View page represents the core visual feature of the AI-Based Travel Agent. After the route is loaded, the MapLibre GL map displays a continuous polyline following the exact geographic path of the selected train route from the source to the destination station. All intermediate stations along the route are rendered as circular markers on the map, with major junction stations shown in a distinct color and size to help passengers identify important stops.

The map automatically fits its viewport to encompass the entire route using turf.js bounding box calculations, with appropriate padding to keep the sidebar visible. As the journey progresses — either in simulation mode or via live GPS tracking — a train icon moves along the route polyline, and the camera smoothly follows the train position with a dynamic pitch and zoom level that provides a cinematic journey view. Stations that the train has already passed are visually dimmed, while the approaching station is highlighted with a pulsing animation.

Passengers can interact with any station marker to view a popup displaying the station name, code, state, and scheduled arrival and departure times. The Journey Status card in the sidebar shows the current estimated distance to the destination, the scheduled arrival time, the dynamically estimated arrival time based on current position, and a delay indicator showing whether the train is on time or running late. A 'LIVE' badge appears in the status card when GPS tracking is active.

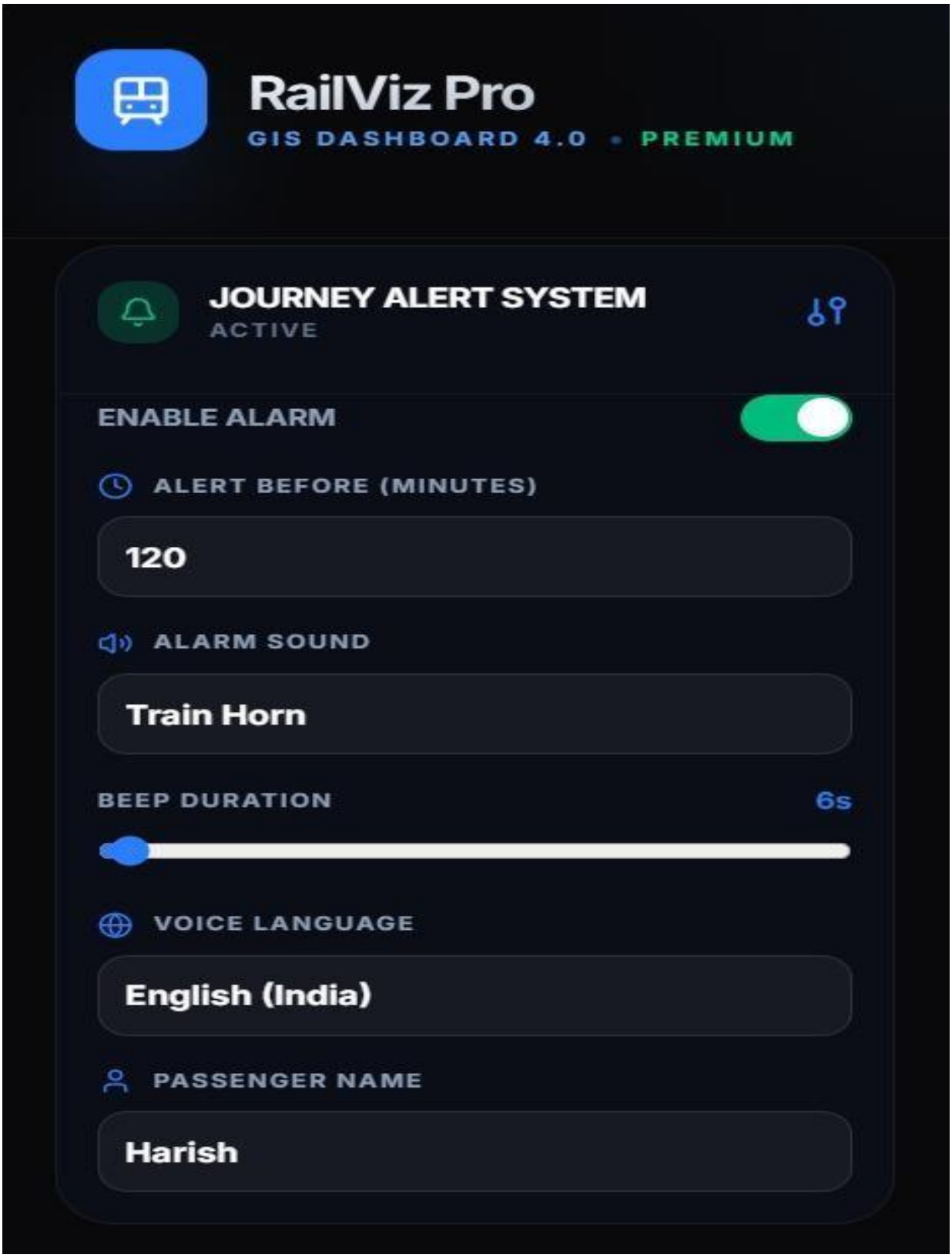


Figure 1.3 Alarm Settings Panel

The beep sound selector offers three distinct options: Train Horn (a realistic two-tone low-frequency horn blast), Alarm Bell (rapid high-pitched beeps), and Emergency Siren (a sweeping frequency alarm). All sounds are generated programmatically using the Web Audio API, requiring no audio file downloads. A 'Test' button allows passengers to preview the selected sound at the configured duration before starting the journey.

The voice language selector lists supported Indian languages with their native script names alongside the English label. When a language is selected, the system checks available Text-to-Speech voices on the device and selects the closest match. The passenger name field accepts any text input, which is embedded into the wake-up voice message: 'Wake up [Name]! [Station] will arrive at [Time].' A toggle switch at the top of the panel enables or disables the entire alarm system, allowing passengers who prefer manual monitoring to turn it off while still using the route tracking features.

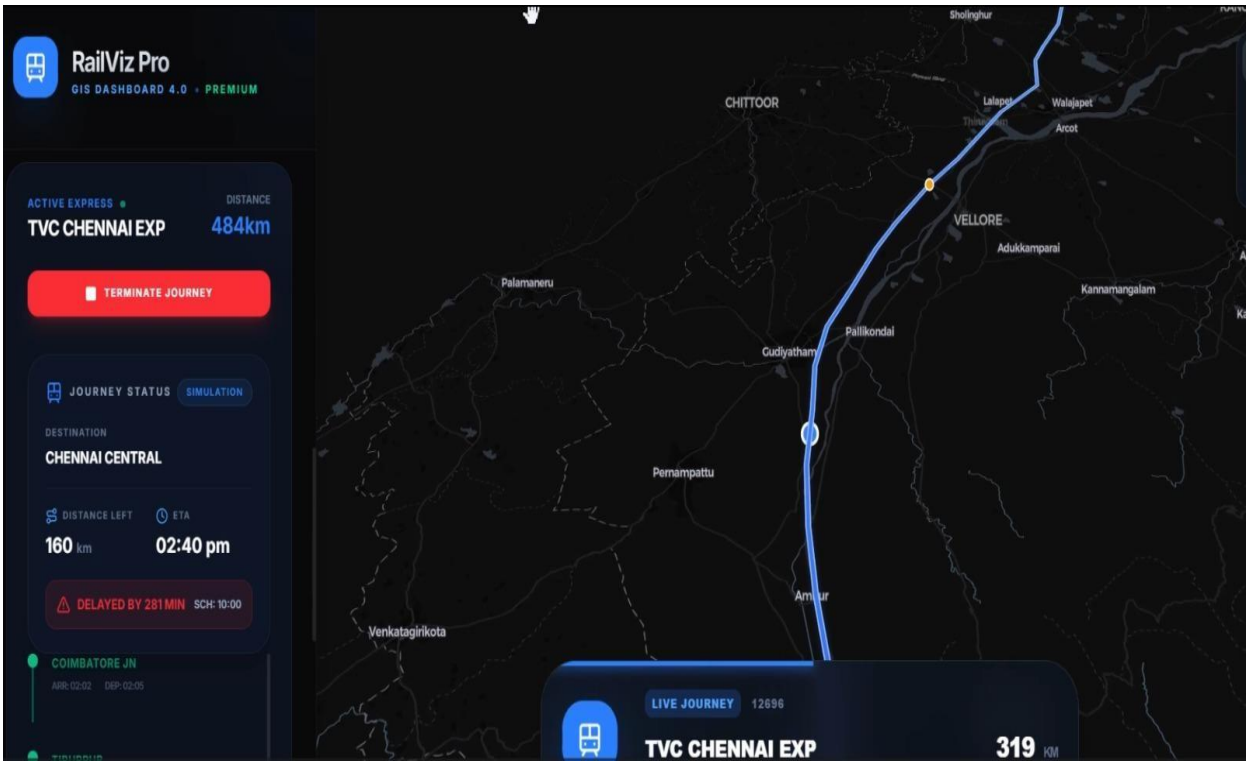


Figure 1.4 Live Journey Tracking Page

The Live Journey Tracking page is the operational heart of the AI-Based Travel Agent during an active journey. Once the passenger presses 'Start Live Journey', the Geolocation API begins streaming real-time GPS coordinates to the application at 5-second intervals with high accuracy enabled. The train icon on the map moves to the passenger's current position, and the camera follows with smooth animation.

The Sleep Detection Module activates automatically when the journey starts. A status indicator in the sidebar shows the current sleep detection state: 'Awake' (green) when recent interactions are detected, 'Drowsy' (yellow) when inactivity exceeds 30 minutes, and 'Sleeping Detected' (red) when inactivity exceeds the configured threshold. When the sleeping state is detected and the alarm trigger threshold is met, the Stage 2 alarm fires immediately without waiting for the Stage 1 countdown to expire.

The full-screen alarm overlay, when activated, covers the entire screen with a flashing red background and displays a large wake-up message with the passenger's name, the destination station, and the estimated arrival time. A persistent 'TAP TO DISMISS' button at the bottom of the overlay stops the beep alarm and voice message when tapped. The overlay cannot be dismissed by pressing the back button, ensuring the passenger consciously acknowledges the alert before it deactivates.

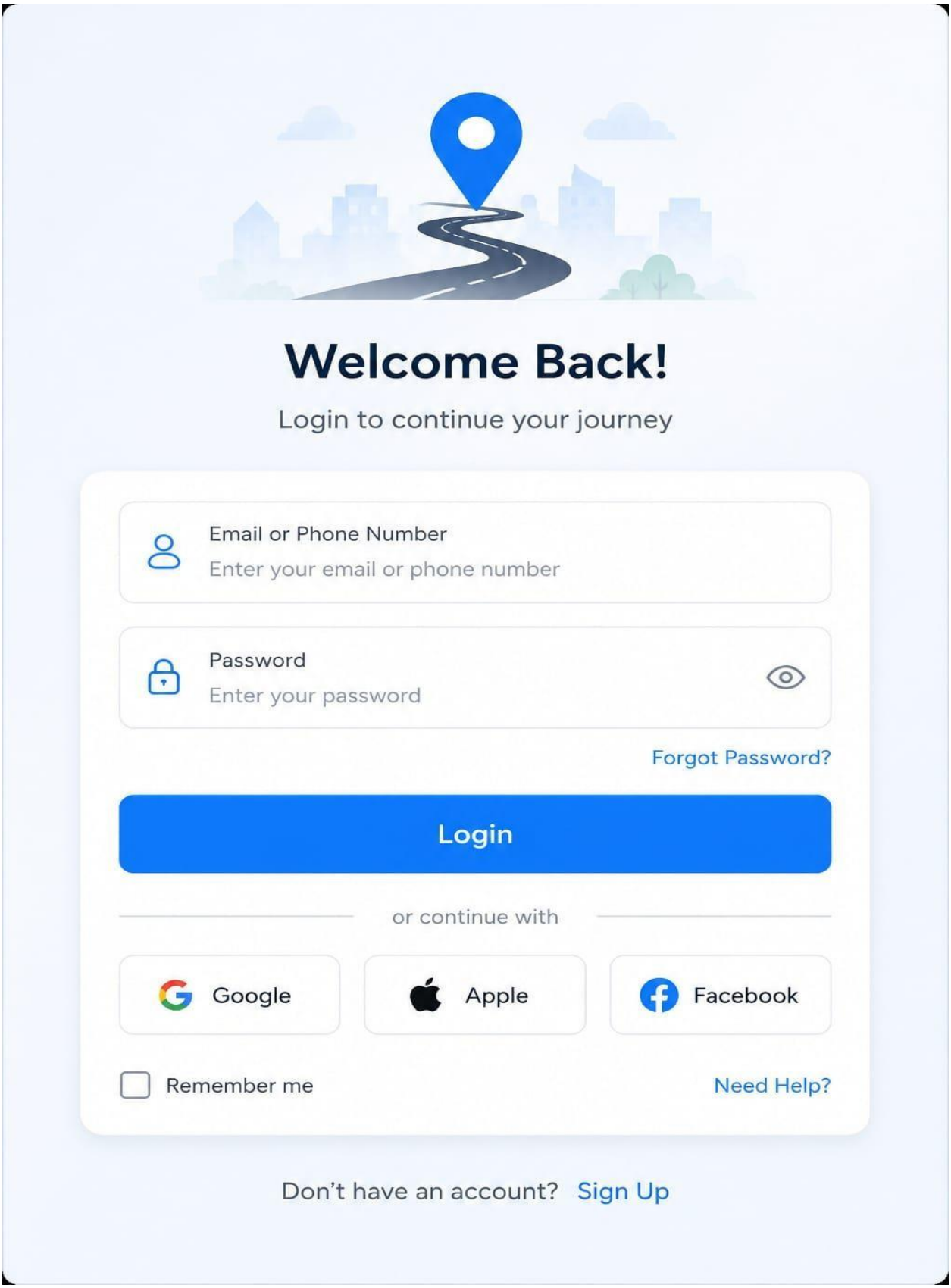


Figure 1.5 Login Page

The Login Page serves as the secure entry point for passengers to access the AI-Based Travel Agent. It is designed with a clean, dark-themed interface consistent with the overall app aesthetic. The page presents fields for email address and password, with a 'Show/Hide Password' toggle for convenience. New passengers can register using the 'Create Account' link, which redirects to a registration form requesting a full name, email, and password.

Authentication is handled securely with encrypted credential verification. Once logged in, the passenger's name is stored and automatically populated in the alarm settings as the default wake-up name. User preferences including alarm settings and recent route history are saved to their profile and restored on subsequent sessions, eliminating the need to reconfigure settings for regular commuters who travel the same route repeatedly. The interface also supports quick access via saved session tokens, so regular users are automatically logged in when they reopen the app.

APPENDIX-2

// Sleep Detection Hook — useAlarmSystem.ts (Core Logic)

```
import { useRef, useEffect, useState, useCallback } from 'react';
import * as turf from '@turf/turf';

export function useAlarmSystem(destStation, routePoints, alarmSettings) {
  const lastInteractionRef = useRef(Date.now()); const
  [alarmStage, setAlarmStage] = useState(0);

  useEffect(() => {
    const updateInteraction = () => { lastInteractionRef.current = Date.now();
    };
    ['touchstart','click','keydown','mousemove'].forEach(e =>
      document.addEventListener(e, updateInteraction));
    document.addEventListener('visibilitychange', () => { if
      (document.visibilityState === 'visible')
        updateInteraction();
  });
  }, []);

  const checkAlarm = useCallback((currentPos) => { const dist =
    turf.distance(
      turf.point(currentPos), turf.point([destStation.longitude,
      destStation.latitude])
    );
    const inactiveMins = (Date.now()-
lastInteractionRef.current)/60000;
    const isSleeping = inactiveMins >= alarmSettings.sleepThreshold;
    const threshold = (alarmSettings.alertMinutes/60)*55; if (dist <=
    threshold) {
      if (isSleeping) setAlarmStage(2); else
      setAlarmStage(1);
    }
  }, [destStation, alarmSettings]); return {
  alarmStage, checkAlarm };
```